DATA ENGINEERING & CLOUD · LESSON 4 OF 15

Data Modelling: Conceptual, Logical and Physical Models, Star and Snowflake Schemas, and SCD Types 1 to 3

Model data at three levels, from business concepts to physical tables. Build star and snowflake schemas, and handle changing dimension values with SCD types 1, 2 and 3, including a working SCD Type 2 load that keeps history and marks the current version.

Gap-fill lesson | ✦ 2026 | Intermediate | ⏱ 150 min

By the end of this lesson you can

- Describe conceptual, logical and physical data models and what each is for
- Build a star schema with a fact table and dimensions with surrogate keys
- Compare star and snowflake designs
- Implement SCD Type 1, Type 2 and Type 3 and explain when each applies

Datasets used

`customers.csv` `superstore_sales.csv`

Three levels of model

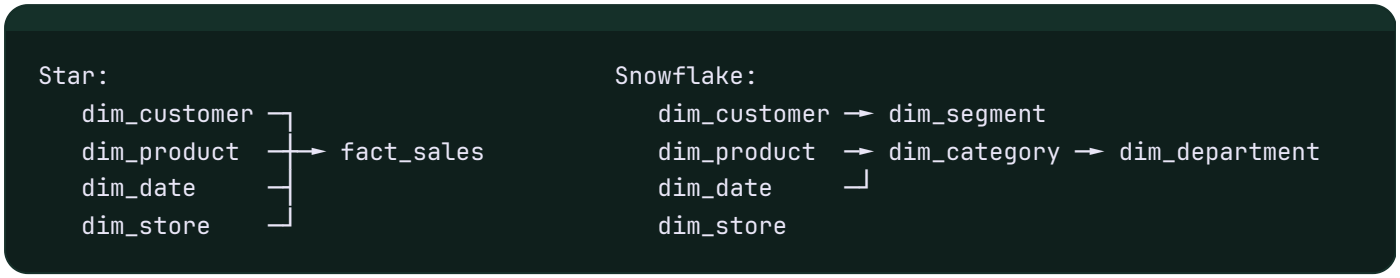
A data model describes what the data means and how it is stored. Designers work at three levels:

Level	Question it answers	Contents	Audience
Conceptual	What are the things the business cares about?	Entities (customer, order, product) and relationships	Business and analysts
Logical	What information does each entity hold?	Attributes, keys, relationships, rules, no technology	Analysts and engineers
Physical	How is it stored in this database?	Tables, types, indexes, partitions	Engineers

Start at the conceptual level with the business, move to logical, and finish at physical. A physical model built without the business concepts stores what the source happens to contain, not what the business needs.

Star and snowflake schemas

A **fact table** records measurable events (a sale, a shipment, a payment) with numeric measures and keys to the dimensions. A **dimension table** describes the context (who, what, where, when). A **star schema** keeps each dimension flat, with one table per dimension. A **snowflake schema** normalises dimensions into sub-tables.



	Star	Snowflake
Query simplicity	Simple, fewer joins	More joins
Storage	Some repetition in dimensions	Less repetition
Typical use	Reporting and BI	Very large or shared hierarchies
Maintenance	One table per dimension	Many small tables

Most reporting warehouses use stars, because the cost of repeating a few text values is small and the queries are much easier to write and understand.

Surrogate keys

A **surrogate key** is a system-generated integer with no business meaning. Facts point to surrogates, not to source keys. This keeps joins small and fast, and insulates the warehouse from source changes (a customer's email can change; the surrogate key does not).

Building a star schema

The block below builds a customer dimension with a surrogate key and a fact table pointing to it, from the course's customer and sales data.

Python

```python
import pandas as pd
import duckdb

con = duckdb.connect()
customers = pd.read_csv("customers.csv")
sales = pd.read_csv("superstore_sales.csv", parse_dates=["order_date"])
con.register("customers_src", customers)
con.register("sales_src", sales)

con.execute("""
    CREATE TABLE dim_customer AS
    SELECT ROW_NUMBER() OVER (ORDER BY customer_id) AS customer_key,
           customer_id, customer_name, segment, region
    FROM customers_src
""")
con.execute("""
    CREATE TABLE fact_sales AS
    SELECT s.row_id AS sale_id, d.customer_key, s.order_date, s.sales, s.profit, s.quantity
    FROM sales_src s
    JOIN dim_customer d ON d.customer_id = s.customer_id
""")

print(con.execute("SELECT COUNT(*) AS facts, COUNT(DISTINCT customer_key) AS customers FROM fact_sales").fetchdf())
print(con.execute("SELECT segment, ROUND(SUM(sales)) AS sales FROM fact_sales f JOIN dim_customer d USING (customer_key) GROUP BY segment ORDER BY sales DESC").fetchdf())
```

OUTPUT

```
   facts  customers
0   6323        785
       segment       sales
0     Consumer  98603292.0
1    Corporate  59113619.0
2  Home Office  43696658.0
```

The fact table holds only keys and numbers; the text lives in the dimension. A report that groups by segment joins once, on an integer.

Slowly changing dimensions

Dimensions change: a customer moves region, changes segment, or changes name. The question is what the reports should show. The answer is an **SCD type**.

Type	Behaviour	History kept?	Use when
Type 1	Overwrite the old value	No	Corrections, where the old value was wrong
Type 2	Add a new row with validity dates and a current flag	Yes, full	The business needs history as it was at the time
Type 3	Add a column for the previous value	One previous value only	Only the last change matters

SCD Type 1: overwrite

Python

```
dim = pd.DataFrame({"customer_id": ["C1"], "region": ["North"]})
# The region was keyed wrongly: overwrite it in place, keeping no history
dim.loc[dim["customer_id"] == "C1", "region"] = "South"
print(dim)
```

OUTPUT

```
  customer_id region
0          C1  South
```

Type 1 loses the old value. Use it only when the old value was an error.

SCD Type 2: keep history

Type 2 closes the old row and inserts a new one, so that every fact joins to the version that was current on its date.

Python

```
import pandas as pd

dim = pd.DataFrame([
    {"customer_key": 1, "customer_id": "C1", "region": "North",
     "valid_from": pd.Timestamp("2024-01-01"), "valid_to": pd.Timestamp("9999-12-31"),
"is_current": True},
])
next_key = dim["customer_key"].max() + 1

def apply_scd2(dim, customer_id, new_region, change_date):
    global next_key
    current = dim[(dim["customer_id"] == customer_id) & dim["is_current"]]
    if current.empty or current.iloc[0]["region"] == new_region:
        return dim                                    # no change: nothing to do
    idx = current.index[0]
    dim.loc[idx, "valid_to"] = change_date - pd.Timedelta(days=1)
    dim.loc[idx, "is_current"] = False
    new_row = {"customer_key": next_key, "customer_id": customer_id, "region": new_region,
               "valid_from": change_date, "valid_to": pd.Timestamp("9999-12-31"),
"is_current": True}
    next_key += 1
    return pd.concat([dim, pd.DataFrame([new_row])], ignore_index=True)

dim = apply_scd2(dim, "C1", "South", pd.Timestamp("2025-06-01"))
dim = apply_scd2(dim, "C1", "South", pd.Timestamp("2025-06-01"))   # rerun: no duplicate
version
print(dim)
```

OUTPUT

```
   customer_key customer_id region valid_from              valid_to  is_current
0             1          C1  North 2024-01-01  2025-05-31 00:00:00       False
1             2          C1  South 2025-06-01  9999-12-31 00:00:00        True
```

The rerun creates no duplicate, because the current row already has the new region. A fact dated March 2025 joins to the North version (key 1); a fact dated July 2025 joins to the South version (key 2). That is what "as it was at the time" means.

SCD Type 3: one previous value

Python

```
dim = pd.DataFrame({"customer_id": ["C1"], "region_current": ["South"], "region_previous":
["North"]})
print(dim)
```

OUTPUT

```
  customer_id region_current region_previous
0          C1          South           North
```

Type 3 keeps a current and a previous column. It is simple and answers "what changed last" questions, but it cannot show a third region.

CASE STUDY

Which version of the customer?

A regional manager asks why last year's North sales now appear in South. The dimension was overwritten with Type 1. The fix is to use Type 2 for region, so that last year's facts keep joining to the North version, and to document the change for the team.

::::warn Overwriting history by accident Many pipelines default to Type 1 because it is easiest. If the business ever asks "what did this look like then?", Type 1 has already destroyed the answer. Decide the SCD type for each attribute with the business, and write the decision down. ::::

INTERVIEW CORNER

When would you use Type 2 rather than Type 1?

When the business needs reports to reflect the attribute as it was at the time of each event: a customer's region for sales, a product's price band for margin, an employee's department for headcount history. Type 1 is for corrections of errors, not for real changes.

::::key What to remember - Model in three levels: conceptual, logical and physical. - A star schema has a fact table and flat dimensions; a snowflake normalises them. - Use surrogate keys so facts join to stable integers. - SCD Type 1 overwrites, Type 2 keeps full history with validity dates, Type 3 keeps one previous value. - Choose the SCD type per attribute with the business. ::::

Quick quiz

5 questions · tap an answer to check it

1. What does a fact table primarily contain?

A Long text descriptions

B Measures and keys to the dimensions

C Only dates

D Passwords

Answer: B. Facts hold events' numbers and the keys that describe them.

2. Why use a surrogate key rather than the source key?

A Source keys are not allowed in databases

B It keeps joins small and insulates the warehouse from source key changes

C It removes the need for dimensions

D It makes the data larger

Answer: B. Surrogates are stable and integer-based.

3. A customer moves from North to South, and reports must show last year's sales in North. Which SCD type fits?

A Type 1

B Type 2

C No SCD is needed

D Type 0

Answer: B. Type 2 keeps the earlier version with validity dates.

4. After an SCD Type 2 change, how many rows does the customer have?

A One

B Two or more, one per version, with one marked current

C Zero

D Always one hundred

Answer: B. Each version is a row with validity dates.

5. What is the weakness of Type 3?

A It is slow

B It keeps only one previous value, so it cannot show a longer history

C It needs no keys

D It deletes data

Answer: B. Type 3 suits "last change" questions only.

EXERCISE **Add a Type 2 change to the customer dimension**

Using the Type 2 function in this lesson, record three region changes for customer C1 (North to South on 1 June 2025, South to East on 1 September 2025, East back to North on 1 December 2025). Print the dimension and then write one query that returns the region a sale dated 15 October 2025 should join to.

Show solution (worked answer)

Expected: three versions after the changes, each with its validity window, and the sale of 15 October 2025 joins to the South version. Use `valid_from ⩽ date AND date ⩽ valid_to` for the join.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.